Herald

Protocol Research Dossier — Index

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (no implementation yet)
Status summary	Initial dossier covering the protocols Herald should consider adding beyond REST: MCP, A2A (now also home of ACP), legacy IBM ACP, OpenAI Realtime, gRPC streaming, WebSocket, SSE, Webhooks, CloudEvents, MQTT, AMQP 1.0, Kafka, NATS JetStream. Verdict matrix below. Each protocol has its own deep-dive document.
Issues	open-questions only — see §3 "Top open questions for operator"
Continuation	Operator decides which protocols to greenlight for Wave 4 (post Wave 3b). Once greenlit, the per-protocol doc's §4 step-by-step implementation guide opens a new HRD-NNN block.

Constitutional anchors

- **§107 anti-bluff** — every protocol document MUST cite primary sources (spec URLs, RFC numbers, GitHub repos at specific commit/version pins at the time of research). PASS-by-grep without runtime evidence is forbidden when implementation begins. Each per-protocol doc carries a §5 "anti-bluff testing strategy" that defines the physical-proof bar for that protocol.
- **§11.4.74 catalogue-check** — for each protocol, we record whether `vasic-digital` or `HelixDevelopment` already hosts a reusable module. No-match protocols are tagged for vendor-as-Herald-internal (with submodule, per Helix §3) or stdlib usage.
- **§11.4.61** — these are tracked docs. HTML/PDF/DOCX siblings to be regenerated via `scripts/export_docs.sh docs/research/protocols/*.md` after every edit.
- **§106 spec-change rule** — these docs live under `docs/research/`, NOT `docs/specs/`. Changes here do NOT trigger the §106 implementation-ripple. Once the operator decides to adopt a protocol, the corresponding spec V3 section (§4x.x) is amended, and only THEN does §106 kick in.

Table of contents

§1. Scope and methodology

The operator's mandate (2026-05-22):

> "Investigate (do deep web research) all we need to know and do step by step in order for Herald and all its Flavor implementations to have full implementations and integrations besides REST API with the following Protocols: MCP, ACP — for work with models (LLMs), any other protocols we should support and same as for these two, the comprehensive in depth research and step by step implementation guide. We MUST have too comprehensive in-depth testing strategy which when applied will produce ALWAYS proofs (and physical proofs) of all integrations fully working in full anti-bluff manner."

Constraint: research only. No code changes outside `docs/research/protocols/`. The operator decides what gets implemented.

Methodology:

1. WebSearch + WebFetch against primary sources only (modelcontextprotocol.io, github.com/a2aproject, ibm.com/research, etc.). Each citation has a fetch date.
2. Catalogue-check (§11.4.74) on `vasic-digital` and `HelixDevelopment` orgs.
3. Map every protocol to Herald's architecture: which flavor binary would surface it, which `commons_*` layer would host the shared code, how `commons_auth` + JWT integrates, how `commons_tls` certs flow through, how Brotli + TOON apply (where they apply), how CloudEvents envelopes propagate.
4. Write the §5 anti-bluff testing strategy keyed to Herald's existing 14-invariant `e2e_bluff_hunt.sh` framework — each protocol gets its own invariant range (Enn–Enn) and mutation gate sketches.

§2. Verdict matrix

Protocol	Verdict	Herald flavor(s)	Replaces REST?	Priority
MCP (Model Context Protocol)	**adopt**	every flavor; new `commons_mcp/` module; flavor binaries expose MCP-server + consume MCP-client	adds (does not replace)	**P0 — first**
A2A (Agent2Agent, Linux Foundation)	**adopt**	every flavor; new `commons_a2a/` module; pherald + cherald + sherald expose A2A AgentCards	adds	**P1 — second**
		(n/a — adopt A2A instead)	n/a	

Protocol	Verdict	Herald flavor(s)	Replaces REST?	Priority
ACP (IBM Agent Communication Protocol)	**observe-only — merged into A2A 2025-08**			rejected as standalone — covered by A2A
CloudEvents (CNCF)	**adopt — already partially adopted**	every flavor (Herald already uses CloudEvents envelopes per `commons.CloudEventEnvelope`); formalize HTTP + Kafka + JSON bindings	extends REST	**P0 — finalize existing**
Webhook (Standard-Webhooks)	**adopt**	pherald (ingest) + every flavor (outbound delivery to user webhooks)	extends REST	**P0**
gRPC streaming	**adopt — high-throughput surface**	pherald (ingest) + cherald (compliance pull, large datasets)	adds — coexists with REST	**P1**
WebSocket	**adopt — interactive surface**	pherald (Telegram-like bots), cherald (live compliance state), iherald (interactive incident chat)	adds	**P2**
SSE (Server-Sent Events, pure HTTP)	**adopt — read-only stream**	cherald + sherald + scherald (status pulls for dashboards)	extends REST	**P1**
OpenAI Realtime API	**adopt as DISPATCHER (client only)**	dispatched-by every flavor via `commons_messaging/dispatch/openai_realtime/` (new)	n/a — outbound LLM dispatch	**P2**
MQTT 5.0	**adopt — IoT ingress**	pherald (ingest) + a future `mherald` flavor for monitoring	adds (event ingress)	**P3 — opt-in**
AMQP 1.0	**adopt — broker ingress**	pherald (ingest from RabbitMQ/Solace)	adds (event ingress)	**P3 — opt-in**
Apache Kafka	**adopt — log ingress**	pherald (ingest); CloudEvents-over-Kafka via `cloudevents/sdk-go/protocol/kafka_sarama`	adds (event ingress)	**P3 — opt-in**
NATS JetStream	**observe — alternative to Kafka**	pherald (ingest) — choose Kafka OR NATS, not both, in MVP	adds (event ingress)	**P4 — alternative**
WebRTC	**REJECT — out of scope**	—	—	rejected (peer-to-peer voice/video is not Herald's mission)
FIPA-ACL (legacy)	**REJECT — historical**	—	—	rejected (1990s multi-agent comm; not the "ACP"

Protocol	Verdict	Herald flavor(s)	Replaces REST?	Priority
				the operator means)

Legend:

- **P0** — recommended for the first protocol wave (Wave 4); high payoff, well-specified, ecosystem-stable.
- **P1** — recommended for second wave; complements P0 by closing specific gaps (high-throughput, read-only streaming, agent interop).
- **P2** — opt-in additions; useful for specific flavor binaries.
- **P3** — operator-decides-per-deployment; useful only if Herald is fronting a broker.
- **P4** — alternative; pick one, not both.

§3. Top open questions for operator

These are the highest-leverage decisions the operator needs to make BEFORE Wave 4 implementation can be planned:

1. **MCP role — server, client, or both?** Does Herald **EXPOSE** MCP servers (i.e. let LLM agents call into Herald to fetch tenant events / safety state / compliance posture as resources) **AND/OR CONSUME** MCP clients (i.e. dispatch outbound notifications to MCP-enabled targets like Claude Desktop, Cursor, etc.)? The two surfaces have very different threat models and code layouts. Recommendation: BOTH, but server-first because it unlocks the highest-value use case (LLMs querying Herald for live posture).

2. **A2A vs MCP overlap — which is the canonical "agent surface"?** A2A and MCP both let agents talk to Herald. MCP is "LLM ? tool" (resource/tool/prompt vocab). A2A is "agent ? agent" (task/artifact vocab). Herald's mission is event fan-out, which maps onto BOTH: an event fan-out target can be either a tool call (MCP) or a delegated task (A2A). Recommendation: implement MCP first; add A2A in parallel only if the operator already has an A2A-native ecosystem (Google ADK, BeeAI, etc.). DO NOT block MCP on A2A.

3. **gRPC for inter-flavor communication?** Right now Herald flavors talk to each other (where they do) via HTTP REST + JWT. Should we add a gRPC surface for high-volume internal fan-out (e.g. pherald → sherald event hand-off)? Tradeoff: gRPC saves bytes + adds streaming, but doubles the maintenance surface and breaks browser-friendliness. Recommendation: defer to operator; the value-add is unclear until profile data shows REST is the bottleneck.

4. **Webhook outbound delivery — Standard-Webhooks spec?** Every Herald flavor will eventually need to emit webhooks to user-configured URLs (notification fan-out target). Adopt the `standard-webhooks.com` spec (Linux Foundation, HMAC-SHA256 + timestamp + replay protection, JWKS-style key rotation)?

5. **CloudEvents transport completeness?** Herald already uses CloudEvents envelopes internally (`commons.CloudEventEnvelope`). Should we formally implement the **HTTP binding** (binary mode + structured mode), the **Kafka binding**, the **AMQP binding**? Each binding is a separate spec. Recommendation: HTTP binding NOW (cheap, no new deps); others gated on §3 question above.

6. **Are MQTT/AMQP/Kafka ingress in-scope for MVP at all?** Herald's spec V3 §10 doesn't list message-broker ingest. If the operator wants Herald to ingest from RabbitMQ / Solace / Kafka, we need to add a §10 amendment. If not, all three are P3-opt-in for far-future expansion.

7. **OpenAI Realtime — first-class dispatcher or research-only?** OpenAI Realtime is WebSocket-based, voice-first, and tightly coupled to OpenAI. It would join `commons_messaging/dispatch/openai_realtime/` alongside the existing Anthropic/Claude Code/Gemini/etc. dispatchers. Recommendation: research-only for now; revisit when voice-notification use cases emerge.

8. **Catalogue-reuse — any existing vasic-digital or HelixDevelopment modules?** Per §11.4.74. Catalogue-check needs running for each protocol; current results are in each protocol's §8. Vendor-as-submodule is the default for no-match.

§4. Recommended adoption sequencing

Assuming the operator greenlights P0+P1 protocols, here is the suggested implementation order (each step is one Herald wave; each maps to a separate HRD-NNN block):

Wave 4a — MCP server surface (highest value, lowest external dependency):

- New module `commons_mcp/` (L1, on top of `commons`).
- Adopt `github.com/modelcontextprotocol/go-sdk` (official Go SDK, Apache 2.0, v1.6.1, 4.6k stars).
- Every flavor binary's `serve` command gains `--mcp-stdio` and `--mcp-http` flags.
- Resources exposed: tenant subscribers, events_processed log, safety state, compliance posture.
- Tools exposed: `notify`, `query_safety`, `list_subscribers`, per flavor.
- New e2e_bluff_hunt invariants E48–E55.

Wave 4b — A2A surface (parallel-track to 4a if operator wants both):

- New module `commons_a2a/` (L1).
- Adopt `github.com/a2aproject/a2a-go/v2` (Linux Foundation, Apache 2.0, v2.3.1, requires Go 1.24.4+).
- Every flavor binary publishes an AgentCard at `/.well-known/agent.json`.
- Tasks: `notify`, `query_state`, `delegate_dispatch`.
- New e2e_bluff_hunt invariants E56–E63.

Wave 4c — CloudEvents HTTP binding formalization:

- Already partially done (envelopes exist); finalize HTTP binding (binary + structured modes) on every `/v1/events` ingest route.
- New e2e_bluff_hunt invariants E64–E66.

Wave 4d — Standard-Webhooks outbound delivery:

- New module `commons_webhook/` (L1) — outbound dispatcher; signs with HMAC-SHA256 + timestamp + JWKS rotation.
- Every flavor's `serve` command gains a webhook delivery worker that pulls from a `webhook_deliveries` table.
- New e2e_bluff_hunt invariants E67–E72.

Wave 4e — gRPC streaming + WebSocket + SSE (interactive surfaces):

- New modules: `commons_grpc/`, `commons_ws/`, `commons_sse/`.
- Wave 4e splits further by flavor depending on operator priority.

Wave 4f and beyond — MQTT, AMQP, Kafka, NATS, OpenAI Realtime — all opt-in per operator.

§5. Rejected with rationale

WebRTC — peer-to-peer voice/video coordination. Herald's mission per §102 of HERALD_CONSTITUTION.md is **event ingestion + notification fan-out**, not media transport. WebRTC's signaling complexity (STUN/TURN/ICE) is a massive operational overhead with no obvious payoff for Herald's use cases. **Investigated → out of scope.**

FIPA-ACL — the legacy "Agent Communication Language" from FIPA (1996). The operator's "ACP" almost certainly means modern IBM ACP (2025), not FIPA-ACL. FIPA-ACL is academic-only and has no current LLM ecosystem alignment. **Investigated → not the protocol the operator means.**

ANP (Agent Network Protocol) — a community-driven sibling to A2A/ACP/MCP, surveyed in arxiv:2505.02279. As of 2026, ANP has not reached the adoption critical mass of A2A or MCP (no flagship LLM vendor backing). Herald can revisit if ANP gains traction. **Investigated → defer (re-evaluate annually).**

XMPP (Extensible Messaging and Presence Protocol) — the old-school IM standard. Federation is its killer feature, but Herald is single-tenant-per-deployment and federation is solved by A2A's AgentCard discovery. No payoff for Herald. **Investigated → rejected.**

STOMP (Simple/Streaming Text Oriented Messaging Protocol) — alternative to AMQP for broker access. Strictly inferior to AMQP 1.0 for Herald's use cases (no equivalent ecosystem). **Investigated → rejected.**

§6. Per-protocol document index

- `MCP.md` — Model Context Protocol (Anthropic, 2024+; current spec 2025-11-25; Apache 2.0; official Go SDK v1.6.1).
- `A2A.md` — Agent2Agent Protocol (Google, contributed to Linux Foundation 2025-06; v1.0 released 2026-03; Apache 2.0; official Go SDK v2.3.1).
- `ACP.md` — IBM's Agent Communication Protocol (May 2025); MERGED INTO A2A as of 2025-08-27. This doc explains the historical context + why Herald should adopt A2A instead.
- `CLOUDEVENTS.md` — CNCF CloudEvents v1.0.2; HTTP / JSON / Kafka / AMQP bindings; Herald already uses CloudEvents envelopes internally.
- `WEBHOOK.md` — Standard-Webhooks spec; HMAC-SHA256 + timestamp + JWKS rotation; CloudEvents payload.
- `GRPC.md` — gRPC streaming; bidirectional + server-streaming; protobuf wire format.
- `WEBSOCKET.md` — RFC 6455; `coder/websocket` (formerly `nhooyr.io/websocket`); chat / interactive flows.
- `SSE.md` — Server-Sent Events; pure HTTP/2; one-way server-to-client streaming.
- `OPENAI_REALTIME.md` — OpenAI Realtime API; WebSocket-based; voice-first; LLM-dispatch surface only.
- `MQTT.md` — MQTT 5.0; Eclipse Paho Go client; pub/sub IoT-style.

- AMQP.md — AMQP 1.0 (RabbitMQ 4.0+ native); broker-mediated message exchange.
- KAFKA.md — Apache Kafka; segmentio/kafka-go vs confluent-kafka-go; CloudEvents-over-Kafka binding.
- NATS.md — NATS JetStream; at-least-once + exactly-once-within-window delivery.

§7. Cross-cutting concerns

Every protocol Herald adopts MUST satisfy the following Herald-architecture invariants. Each per-protocol doc's §3 expands on the protocol-specific shape; this section is the shared contract.

§7.1 Auth — `commons_auth` JWT first, protocol-native auth second

Herald has a JWT verifier in `commons_auth/` (Wave 3a). Every new protocol surface MUST integrate via one of:

1. **JWT-over-protocol** — the protocol carries the Bearer token (e.g. MCP via OAuth 2.0 client flow; A2A via HTTP Authorization header; webhooks via custom `Herald-JWT` header). Preferred.
2. **Protocol-native auth + Herald audit log** — for protocols that already have battle-tested auth (MQTT's username/password + TLS client cert; AMQP SASL; gRPC mTLS), Herald wraps the protocol-native identity into a synthetic JWT for audit log compatibility.

In both cases, the protocol surface MUST emit the same `events_processed` row + `audit_log` row that REST emits, keyed by `tenant_id` and `subject`.

§7.2 Tenant isolation — `tenant_id` is load-bearing

Every protocol MUST carry `tenant_id` either:

- in the JWT claim (preferred — comes via §7.1.1), or
- in a protocol-specific header (MCP: `X-Herald-Tenant`; A2A: AgentCard metadata; webhook: signed in HMAC payload; Kafka: header `ce-herald-tenant`).

A request without resolvable `tenant_id` MUST be REJECTED at the transport edge. No fallback to "default tenant".

§7.3 Idempotency — Redis SETNX + events_processed

Every inbound message (REST, MCP, A2A, webhook, MQTT, Kafka, etc.) MUST go through the existing idempotency path:

1. Compute `idempotency_key` (from payload or transport metadata).
2. Redis SETNX with TTL.
3. If first-seen → process + insert `events_processed`.
4. If duplicate → no-op + return 202.

Each protocol's §3 defines the protocol-specific `idempotency_key` derivation.

§7.4 OTel propagation — W3C TraceContext on every protocol

Every inbound + outbound message MUST carry a W3C TraceContext (`traceparent` + `tracestate` headers for HTTP-like; binary span context for protobuf/gRPC; `ce-`

`traceparent` extension attribute for CloudEvents). Herald's `commons_observability/` (from `submodules/observability/`) provides the propagator wiring.

§7.5 TLS — `commons_tls` cert reuse

All wire-protocol surfaces (HTTP, gRPC, WebSocket, SSE, MQTT-over-TLS, AMQP-over-TLS, Kafka-over-TLS) reuse Herald's existing dev-cert auto-generation in `commons_tls/`. No protocol may bypass TLS in non-dev environments.

§7.6 Brotli + TOON applicability

- **HTTP-based protocols (REST, MCP-Streamable-HTTP, A2A-JSON-RPC-over-HTTP, webhook, SSE)** — Brotli compression is in scope; TOON is in scope (alternative content-type for AI-consumer endpoints).
- **Binary protocols (gRPC, MQTT, AMQP, Kafka)** — they have their own compression (gzip/snappy/lz4); Brotli is NOT applicable. TOON is also N/A (binary protocols don't carry JSON-shape responses).
- **WebSocket** — Brotli is applicable per-frame; TOON is applicable per-message.

§7.7 Observability — every protocol gets `/metrics` + `/healthz` + `/readyz`

Even on non-HTTP protocols, the FLAVOR binary's existing `:8081/metrics` Prometheus endpoint MUST expose protocol-specific counters: `herald_mcp_requests_total`, `herald_a2a_tasks_total`, `herald_webhook_deliveries_total{status="success|fail|retry"}`, `herald_mqtt_messages_total`, etc.

§7.8 Anti-bluff §107 — physical proof of working end-user behaviour

Per Herald §107 + Helix §11.4: each protocol's `e2e_bluff_hunt.sh` invariant MUST produce **physical evidence** of the protocol working end-to-end against real services. Examples (each per-protocol doc's §5 expands):

- **MCP** — a real `claude-code mcp` client connects to a real `pherald serve --mcp-stdio` process, lists tools, invokes one, asserts the response byte sequence.
- **A2A** — a real `a2a` CLI tool resolves Herald's AgentCard from `https://localhost:7104/.well-known/agent.json`, sends a `message/send` JSON-RPC call, asserts the artifact return.
- **gRPC** — `grpcurl` invokes the streaming endpoint, asserts N messages return in stream-order.
- **WebSocket** — `websocat` opens a connection, asserts ping/pong + N frames.
- **MQTT** — a real `mosquitto_pub` against a real `mosquitto` broker (booted on-demand by `containers/`), Herald subscribes, asserts message receive.

No metadata-only PASS. No grep-only PASS. No "absence of error" PASS. The anti-bluff bar is "the end user can use this feature against real services."

§8. References

(All fetched 2026-05-22.)

- Model Context Protocol — https://modelcontextprotocol.io/specification/2025-11-25 ; https://github.com/modelcontextprotocol/modelcontextprotocol (8.2k stars); https://github.com/modelcontextprotocol/go-sdk (v1.6.1, 4.6k stars).
- A2A Protocol — https://github.com/a2aproject/A2A (23.9k stars, v1.0.0 2026-03-12); https://github.com/a2aproject/a2a-go (v2.3.1, 381 stars); https://google.github.io/A2A/.
- IBM ACP — https://research.ibm.com/projects/agent-communication-protocol ; https://github.com/i-am-bee/acp (1k+ stars; archived 2025-08-27, MERGED INTO A2A).
- CloudEvents — https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md ; https://github.com/cloudevents/sdk-go.
- Standard Webhooks — https://github.com/standard-webhooks/standard-webhooks ; https://www.webhooks.fyi/security/hmac.
- gRPC — https://grpc.io/docs/languages/go/basics/ ; https://github.com/grpc/grpc-go.
- WebSocket (RFC 6455) — https://datatracker.ietf.org/doc/html/rfc6455 ; https://github.com/coder/websocket (formerly nhooyr.io/websocket).
- Server-Sent Events — https://html.spec.whatwg.org/multipage/server-sent-events.html ; https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events.
- OpenAI Realtime — https://developers.openai.com/api/docs/guides/realtime-websocket ; https://platform.openai.com/docs/guides/realtime.
- MQTT 5.0 — https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html ; https://github.com/eclipse-paho/paho.mqtt.golang.
- AMQP 1.0 — https://www.amqp.org/specification/1.0/amqp-org-download ; https://www.rabbitmq.com/docs/amqp ; https://github.com/rabbitmq/amqp091-go.
- Apache Kafka — https://kafka.apache.org/protocol ; https://github.com/segmentio/kafka-go ; https://github.com/confluentinc/confluent-kafka-go.
- NATS JetStream — https://docs.nats.io/nats-concepts/jetstream ; https://github.com/nats-io/nats.go.
- Survey of agent interop protocols — arXiv:2505.02279 ; arXiv:2510.13821 ; arXiv:2602.15055.